```
    { name: 'hal_9000', number: 'disconnected', relationship:
'friend' }
    { name: 'freddy', number: '555-5554', relationship: 'friend' }
    { name: 'T-800', number: '555-5555', relationship: 'colleague'
}
  ]
```

If the phone book is an event emitter, then it's still callback-driven, except that the callback is called repeatedly as new data arrives:

```
phonebook.on 'data', callback
```

The sequence of values produced by an event emitter is an array; you just don't have all of the values yet. Think of the array of keyboard commands that were produced during the running game of Pong—it was also produced by an event emitter (a user):

```
[ UP, DOWN, DOWN, DOWN, UP, DOWN, UP, UP, UP ]
```

An event emitter generating the phone book doesn't change what the data looks like. Instead of letting the source of the data dictate how you use it, decide on the interface you want first, and then figure out how to make it work.

9.4.3. Composing event streams

Suppose the phone book information is arriving asynchronously, with each subsequent chunk arriving as it becomes available. The source of the data is an event emitter named `phonebookEmitter` that may or may not stop producing more events. Now suppose that you want to filter the phone book to just friends. The interface that you want is one that matches what you use for an array:

```
phonebook = \
withEvents(phonebookEmitter)
.filter(relationship 'friend')
```

You start by trying this on a different source of events so that you can control what's happening. Create your own event emitter that emits numbers: